VOL. 06

Angular

Components, signals, routing and forms, from first app to signal forms

Source	angular.dev/tutorials
Generated	2026-10-10
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 06 - Angular

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8

Chapter map 8

Chapter 1 - What is Angular and the Angular CLI setup 9

1.1 Summary 9

1.2 Concepts 9

The parts of the framework 10

Creating and running a project 10

What the scaffolding produces 10

The bootstrap in `main.ts` 10

1.3 Examples 11

1.4 Common mistakes and best practices 12

1.5 Practice exercises 12

Solutions 12

1.6 Mini test 12

Answer key and explanations 13

Chapter 2 - Components and the component class 13

2.1 Summary 13

2.2 Concepts 14

Anatomy of a component 14

Template syntax at a glance 14

Lifecycle hooks you use first 14

2.3 Examples 14

2.4 Common mistakes and best practices 16

2.5 Practice exercises 16

Solutions 16

2.6 Mini test 17

Answer key and explanations 17

Chapter 3 - Composing components and data flow 17

3.1 Summary 17

3.2 Concepts 18

The data flow 18

Declaring inputs and outputs 18

Transforms and aliases . . . 18
3.3 Examples . . . 19
3.4 Common mistakes and best practices . . . 21
3.5 Practice exercises . . . 21
Solutions . . . 21
3.6 Mini test . . . 21
Answer key and explanations . . . 22
Chapter 4 - Templates and control flow . . . 22
4.1 Summary . . . 22
4.2 Concepts . . . 22
Binding kinds . . . 22
Conditionals with `@if` . . . 23
Loops with `@for` . . . 23
Switch . . . 23
4.3 Examples . . . 23
4.4 Common mistakes and best practices . . . 24
4.5 Practice exercises . . . 25
Solutions . . . 25
4.6 Mini test . . . 25
Answer key and explanations . . . 25
Chapter 5 - Dependency injection and services . . . 26
5.1 Summary . . . 26
5.2 Concepts . . . 26
A root service . . . 26
Injecting . . . 27
A token for plain values . . . 27
Scoping a service to a component . . . 27
5.3 Examples . . . 28
5.4 Common mistakes and best practices . . . 29
5.5 Practice exercises . . . 30
Solutions . . . 30
5.6 Mini test . . . 30
Answer key and explanations . . . 30
Chapter 6 - Routing and navigation . . . 31
6.1 Summary . . . 31

6.2 Concepts . . . 31
The route table . . . 31
The shell . . . 31
Reading parameters . . . 32
Navigating from code . . . 32
6.3 Examples . . . 32
6.4 Common mistakes and best practices . . . 33
6.5 Practice exercises . . . 34
Solutions . . . 34
6.6 Mini test . . . 34
Answer key and explanations . . . 35
Chapter 7 - Forms: template-driven and reactive . . . 35
7.1 Summary . . . 35
7.2 Concepts . . . 35
Reactive building blocks . . . 35
A group with validators . . . 35
The template . . . 36
Template-driven alternative . . . 36
7.3 Examples . . . 36
7.4 Common mistakes and best practices . . . 38
7.5 Practice exercises . . . 39
Solutions . . . 39
7.6 Mini test . . . 39
Answer key and explanations . . . 39
Chapter 8 - HTTP client and async data access . . . 40
8.1 Summary . . . 40
8.2 Concepts . . . 40
Registering the client . . . 40
A typed data service . . . 40
Consuming it in a component . . . 41
Error handling . . . 41
8.3 Examples . . . 42
8.4 Common mistakes and best practices . . . 43
8.5 Practice exercises . . . 44
Solutions . . . 44

8.6 Mini test . . . 44
Answer key and explanations . . . 44
Chapter 9 - Signals: state, computed, linked and effects . . . 45
9.1 Summary . . . 45
9.2 Concepts . . . 45
Creating and reading . . . 45
Writable signal API . . . 45
Equality and options . . . 46
linkedSignal . . . 46
effect . . . 46
Signal queries . . . 46
9.3 Examples . . . 47
9.4 Common mistakes and best practices . . . 49
9.5 Practice exercises . . . 49
Solutions . . . 49
9.6 Mini test . . . 49
Answer key and explanations . . . 50
Chapter 10 - Deferrable views and image optimization . . . 50
10.1 Summary . . . 50
10.2 Concepts . . . 50
The four blocks . . . 50
Triggers . . . 51
Images . . . 51
10.3 Examples . . . 51
10.4 Common mistakes and best practices . . . 52
10.5 Practice exercises . . . 53
Solutions . . . 53
10.6 Mini test . . . 53
Answer key and explanations . . . 54
Chapter 11 - Pipes and data formatting . . . 54
11.1 Summary . . . 54
11.2 Concepts . . . 54
Syntax . . . 54
A custom pipe . . . 55
Purity . . . 55

11.3 Examples . . . 55
11.4 Common mistakes and best practices . . . 57
11.5 Practice exercises . . . 58
Solutions . . . 58
11.6 Mini test . . . 58
Answer key and explanations . . . 59
Chapter 12 - Signal-based forms . . . 59
12.1 Summary . . . 59
12.2 Concepts . . . 59
The model and the form . . . 59
Reading field state . . . 60
Standard rules . . . 60
Submitting . . . 61
12.3 Examples . . . 61
12.4 Common mistakes and best practices . . . 62
12.5 Practice exercises . . . 62
Solutions . . . 63
12.6 Mini test . . . 63
Answer key and explanations . . . 63
Cheat sheet . . . 63
13.1 CLI . . . 63
13.2 Component . . . 64
13.3 Inputs, outputs, models . . . 64
13.4 Control flow . . . 64
13.5 Bindings . . . 64
13.6 Dependency injection . . . 65
13.7 Router . . . 65
13.8 Reactive forms . . . 65
13.9 Signals . . . 65
13.10 Defer and images . . . 66
13.11 Pipes . . . 66
13.12 Signal forms . . . 66
13.13 HTTP . . . 67
Final test . . . 67

14.1 Final test answers 68
Sources 69

How to use this volume

Prerequisites: HTML (Vol. 01), CSS (Vol. 02) and JavaScript (Vol. 03). You should be comfortable with classes, arrow functions, modules, promises and the DOM. No previous Angular experience is assumed.

Estimated study time: about 30 hours, roughly two to three hours per chapter.

How the chapters work: every chapter opens with a summary, explains the concepts with tables and diagrams, shows runnable examples, lists the common mistakes with their fixes, and closes with practice exercises, solutions and a mini test with an answer key. Work through the examples in a real project created with the Angular CLI.

How the code blocks were produced. Three kinds of blocks appear in this book:

- `bash` blocks are CLI commands. Their output changes with the CLI version and your machine, so it is not reproduced; each block says so in a note.
- `ts` blocks that only use `signal`, `computed`, `FormControl`, `Validators` or the built-in pipes were executed with Node v22 and the published packages `@angular/core`, `@angular/common` and `@angular/forms`, all version 22.2.2. The output blocks under them are real captured output.
- `html` blocks and `ts` blocks that define components render in a browser through `ng serve`. They were not executed here, so each carries a note; where the rendered result is described, it is a prediction based on the API and not captured output.

Note conventions: a note inside an example is spelled `NOTE` and uses the comment syntax of the fence language: `// NOTE` in TypeScript, `# NOTE` in bash and `<!-- NOTE ... -->` in HTML.

Version: every API detail was checked against the published packages `@angular/core@22.2.2`, `@angular/common@22.2.2` and `@angular/forms@22.2.2` installed from npm on 2026-10-10. CLI commands target a recent Angular CLI.

How to run the code

shell

```shell
npm install -g @angular/cli     # install the CLI once, globally
ng new study-app --routing      # scaffold a project named study-app
cd study-app
ng serve --open                 # dev server, by default on port 4200
```

Chapter map

#	Chapter	You will learn
1	What is Angular and the CLI setup	the framework's parts, project layout, `ng new`, `ng serve`
2	Components and the component class	`@Component`, templates, class fields, lifecycle
3	Data flow between components	`input()`, `output()`, property and event binding, `model()`

#	Chapter	You will learn
4	Templates and control flow	interpolation, bindings, `@if`, `@for`, `@switch`
5	Dependency injection and services	`@Injectable`, `inject()`, `providedIn`, tokens
6	Routing and navigation	`provideRouter`, routes, `routerLink`, route parameters
7	Forms	template-driven forms and reactive forms with validation
8	HTTP and async data	`HttpClient`, observables, error handling
9	Signals	`signal`, `computed`, `linkedSignal`, `effect`, `model`
10	Deferrable views and images	`@defer`, triggers, `NgOptimizedImage`
11	Pipes	built-in pipes, parameters, custom pipes
12	Signal-based forms	`form()`, field state, validators, `submit()`
-	Cheat sheet	compact reference tables
-	Final test	25 questions with answers

Chapter 1 - What is Angular and the Angular CLI setup

1.1 Summary

- Angular is a component framework: the UI is a tree of components, and each component pairs a TypeScript class with an HTML template.
- The Angular CLI scaffolds, builds and serves a project, so you write app code instead of configuring toolchains.
- A generated project is standalone-component based; `main.ts` calls `bootstrapApplication` with the root component and its providers.
- Angular's npm packages share one version number, and the CLI must match it.
- Node.js LTS is required; the published `@angular/core` 22.2.2 package declares an engines range of Node 22.22.3 or newer, 24.15.0 or newer, or 26 or newer.
- `ng serve` runs a development server with rebuilds; `ng build` writes a production bundle.
- `ng generate` adds components, services and other pieces with the boilerplate already in place.
- The build compiles TypeScript, templates and styles together, and reports template type errors as build errors.

1.2 Concepts

The parts of the framework

```
+-------------------------------------------------------------+
|  application bootstrap  (main.ts, bootstrapApplication)     |
|                                                             |
|  +---------+   +---------+   +-----------+   +-------------+ |
|  | router  |   | forms   |   | http      |   | pipes       | |
|  | @angular/router      | @angular/forms | @angular/common | |
|  +---------+   +---------+   +-----------+   +-------------+ |
|                                                             |
|  +---------+   +---------+   +-----------+   +-------------+ |
|  | signals |   | DI      |   | templates |   | components  | |
|  | @angular/core (signal, computed, effect, inject)          | |
|  +---------+   +---------+   +-----------+   +-------------+ |
+-------------------------------------------------------------+
```

Everything except the router lives in `@angular/core` or `@angular/common`. Forms live in `@angular/forms`, and the router in `@angular/router`.

Creating and running a project

shell

```shell
 # NOTE: output is not reproduced here; it depends on the CLI version.
npm install -g @angular/cli
ng new study-app --routing --style=css
cd study-app
ng serve --open
```

The CLI asks about server-side rendering and analytics when you run `ng new` without `--defaults`; answering them changes the generated files slightly.

What the scaffolding produces

output

```
study-app/
  angular.json          build and serve configuration per project
  package.json          scripts and dependencies
  tsconfig.json         TypeScript compiler options
  src/
    main.ts             bootstrap call
    index.html          host page
    styles.css          global styles
    app/
      app.ts            root component
      app.config.ts     application providers
      app.routes.ts     route table (added by --routing)
```

The bootstrap in `main.ts`

TypeScript

```typescript
import { bootstrapApplication } from '@angular/core';
import { AppComponent } from './app/app';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig);
```

The application config is a plain list of providers:

TypeScript

```typescript
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)],
};
```

1.3 Examples

Example 1: create the project and start the dev server.

shell

```shell
 # NOTE: output is not reproduced here; the CLI prints progress lines and
 # depends on the installed version, package manager and network.
ng new study-app --routing
cd study-app
ng serve --open
```

The dev server prints a local URL (4200 by default) and rebuilds on every file save.

Example 2: generate a component and a service.

shell

```shell
 # NOTE: output is not reproduced here; file names and the exact wording of
 # the summary depend on the CLI version and existing files.
ng generate component housing-list
ng generate service housing
```

Example 3: build for production.

shell

```shell
 # NOTE: output is not reproduced here; timings and chunk names differ per run.
ng build
```

Example 4: the smallest possible standalone component.

TypeScript

```typescript
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  template: `<h1>Hello, Angular</h1>`,
})
export class HelloComponent {}
```

Example 5: render it from the root component.

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
<main>
  <app-hello />
</main>
```

Expected rendering: the page shows the `h1` text `Hello, Angular`.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Installing the CLI as a project dependency	version drift and slow installs	one global install
Ignoring the Node engine warning	builds fail in confusing ways	use the Node version the package asks for
Running `ng serve` outside a project	no `angular.json` is found	run it in the folder that has `angular.json`
Editing files under `node_modules`	changes vanish on reinstall	change your own source
Mixing package versions	runtime and type errors	keep all `@angular/*` on one version

- Read `angular.json` once: it explains how `ng serve` and `ng build` differ.
- Keep the project structure the CLI generates; generators add files where the build expects them.

1.5 Practice exercises

1. Which command creates a project named `shop` with routing enabled?
2. Which file calls `bootstrapApplication` in a fresh project?
3. Where do you register providers such as `provideRouter`?
4. Why should all `@angular/*` packages share one version?
5. Which command starts a development server that rebuilds on save?

Solutions

1. `ng new shop --routing`.
2. `src/main.ts`.
3. In the application config object (`app.config.ts`), passed as the second argument of `bootstrapApplication`.
4. Angular's packages are tested and published as one aligned set; mixing versions produces type and runtime errors that are hard to diagnose.
5. `ng serve`.

1.6 Mini test

1. Multiple choice: which function starts a standalone Angular application? - a) `bootstrapModule` - b) `bootstrapApplication` - c) `platformBrowserDynamic` - d) `ngServe`
2. Multiple choice: which file holds the route table after `ng new --routing`? - a) `app.routes.ts` - b) `app.module.ts` - c) `router.json` - d) `main.ts`
3. True or false: components must be declared inside an `NgModule` in a new project.
4. Short answer: which flag of `ng new` adds routing to the generated app?
5. Multiple choice: what does `ng generate component housing-list` create? - a) only a TypeScript file - b) a component class, template and style file - c) a new project - d) a service
6. True or false: `ng build` starts a watch mode like `ng serve`.
7. Short code question: write the `@Component` decorator for a component with selector `app-card` and an inline template.

Answer key and explanations

1. b - `bootstrapApplication(AppComponent, config)` is the standalone bootstrap API.
2. a - `app.routes.ts` exports the routes array.
3. False - standalone components need no module.
4. `--routing`.
5. b - the class file plus template and style files.
6. False - `ng build` produces an output tree once; `ng serve` watches.
7. `@Component({ selector: 'app-card', template:` <h1>Card</h1> `})`.

Chapter 2 - Components and the component class

2.1 Summary

- `@Component` attaches a selector, a template and styles to a class.
- Since Angular 19 components are standalone by default, so no module is needed.
- The class holds state as fields and behaviour as methods; the template renders them.
- Interpolation `{{ }}` prints expressions; property binding `[prop]` sends values into the DOM or child components.
- `templateUrl`, `styleUrl` and `styleUrls` move the template and styles into separate files; `template` and `styles` keep them inline.
- Lifecycle hooks such as `ngOnInit` and `ngOnDestroy` run at defined points of the component's life.
- Styles are scoped to the component by default (emulated encapsulation).
- One component, one job: keep classes small and compose them into a tree.

2.2 Concepts

Anatomy of a component

TypeScript

```ts
@Component({
  selector: 'app-counter',          // how the template uses it: <app-counter />
  template: `
    <p>Count: {{ count }}</p>
    <button type="button" (click)="increment()">+1</button>
  `,
  styleUrl: './counter.css',        // one stylesheet file for this component
  imports: [CommonModule],          // other components and directives used
})
export class CounterComponent {
  count = 0;

  increment() {
    this.count += 1;
  }
}
```

Template syntax at a glance

Kind	Syntax	Direction
Interpolation	`{{ count + 1 }}`	class to DOM
Property binding	`[src]="imageUrl"`	class to DOM property
Attribute binding	`[attr.aria-label]="label"`	class to DOM attribute
Event binding	`(click)="save()"`	DOM to class
Two-way binding	`[(value)]="name"`	both, with a model input

Lifecycle hooks you use first

```
construct -> ngOnChanges -> ngOnInit -> ... -> ngAfterViewInit
                                  ...
                ngDoCheck (every change detection run)
                                  ...
                          ngOnDestroy
```

Hook	Typical use
`constructor`	assign injected dependencies only
`ngOnInit`	start loading data
`ngOnDestroy`	release subscriptions and timers

Emitting outputs are a class job covered in Chapter 3; writing to the DOM directly is avoided.

2.3 Examples

Example 1: a component with state and a method.

TypeScript

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-greeter',
  template: `<p>{{ greeting() }}</p>`,
})
export class GreeterComponent {
  name = 'Ada';
  greeting() {
    return `Hello, ${this.name}`;
  }
}
```

Example 2: the same component with separate files.

TypeScript

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-greeter',
  templateUrl: './greeter.html',
  styleUrl: './greeter.css',
})
export class GreeterComponent {}
```

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<p>{{ name }} starts with {{ name[0] }}</p>
```

Example 3: lifecycle hooks with a counter.

TypeScript

```ts
import { Component, OnInit, OnDestroy } from '@angular/core';

@Component({
  selector: 'app-ticker',
  template: `<p>{{ seconds }}</p>`,
})
export class TickerComponent implements OnInit, OnDestroy {
  seconds = 0;
  private timer = 0;

  ngOnInit() {
    this.timer = setInterval(() => (this.seconds += 1), 1000);
  }

  ngOnDestroy() {
    clearInterval(this.timer);
  }
}
```

Example 4: interpolation renders expressions, not statements.

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
<p>{{ 2 + 3 }}</p>
<p>{{ name.toUpperCase() }}</p>
```

Expected rendering: `5` and the name in capitals. Statements such as `{{ if (x) { } }}` are a syntax error in templates; use control flow instead (Chapter 4).

Example 5: styles are scoped to the component.

TypeScript

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-badge',
  template: `<span class="tag">new</span>`,
  styles: [`.tag { color: crimson; }`],
})
export class BadgeComponent {}
```

A global `.tag` rule elsewhere does not reach this span, and this rule does not leave the component, because emulated encapsulation rewrites the selectors.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Heavy logic in the constructor	runs before inputs exist	use `ngOnInit`
Forgetting `ngOnDestroy` cleanup	leaks timers and subscriptions	release what you start
Reaching into the DOM from the class	breaks server rendering and tests	bind in the template
One giant component	hard to test and reuse	split into child components
Global styles for component looks	styles leak between components	component `styles` or `styleUrl`

2.5 Practice exercises

1. Write a component `app-hello` whose template shows the class field `title`.
2. Which hook runs once after the first change detection of the inputs?
3. Why is `setInterval` started in the constructor a bad idea?
4. Which decorator field names a single external stylesheet?
5. What does Angular do to the `.tag` selector of Example 5?

Solutions

1. `ts @Component({ selector: 'app-hello', template: <h1>{{ title }}</h1>, }) export class HelloComponent { title = 'Welcome'; }`

2. `ngOnInit` runs after the first `ngOnChanges`; `ngAfterViewInit` runs after the view is initialised. For "after the first change detection pass" the answer is `ngAfterContentInit`-family hooks, so state reading belongs in `ngOnInit`.
3. It starts before the component is fully initialised, and there is no guaranteed cleanup point unless you also implement `ngOnDestroy`.
4. `styleUrl`.
5. It rewrites it with a unique attribute so the rule only matches elements of that component (emulated encapsulation).

2.6 Mini test

1. Multiple choice: which decorator marks a class as a component? - a) `@Injectable` - b) `@Component` - c) `@Directive` - d) `@Pipe`
2. Multiple choice: which entry in `@Component` names the element? - a) `template` - b) `selector` - c) `styleUrl` - d) `imports`
3. True or false: interpolation `{{ }}` can contain any JavaScript statement.
4. Short answer: which array in `@Component` lists the other components this component uses?
5. Multiple choice: where should data loading start? - a) the constructor - b) `ngOnInit` - c) the field initialiser - d) the decorator
6. True or false: component styles apply to the whole document by default.
7. Short code question: bind the class field `imageUrl` to an image's `src`.

Answer key and explanations

1. b - `@Component`.
2. b - `selector`; the template writes it as `<app-...>`.
3. False - only expressions, not statements.
4. `imports`.
5. b - `ngOnInit`, after inputs are set.
6. False - they are scoped to the component by default.
7. `<img [src]="imageUrl" alt="" />`.

Chapter 3 - Composing components and data flow

3.1 Summary

- A parent passes data down through inputs and receives events through outputs.
- `input()` declares a signal input: read it in the template by calling it.
- `input.required()` marks an input that the parent must bind.
- `output()` creates an emitter; the parent listens with an event binding.

- `model()` declares a writable signal that works as an input and an output, so `[(value)]="name"` gives two-way binding.
- Property binding sends values into an element or child; event binding calls a method with `$event`.
- `input()` accepts a transform function, and both `input` and `output` accept an `alias` so the public name can differ from the field name.
- Inputs are read-only from the child's point of view: the child never assigns to them.

3.2 Concepts

The data flow

```
        parent component                    child component
  +-----------------------+          +-----------------------+
  |  selectedId = 2       |  [id]    |  id = input<number>()  |
  |                       | -------> |  label = computed(...) |
  |  onPick($event)       | <------- |  picked = output()     |
  +-----------------------+  (picked)+-----------------------+
```

Declaring inputs and outputs

TypeScript

```ts
import { Component, input, output, model } from '@angular/core';

@Component({
  selector: 'app-listing',
  template: `<h2>{{ title() }}</h2>`,
})
export class ListingComponent {
  title = input.required<string>();        // must be bound by the parent
  highlight = input('');                   // optional, default ''
  selectedId = model(0);                   // two-way, default 0
  picked = output<number>();               // event stream to the parent
  pickedAlias = output({ alias: 'choose' });
}
```

Declaration	Read in class	Bind in parent template
`input()`	`this.name()`	`[name]="value"`
`input.required()`	`this.name()`	`[name]="value"` (compiler requires it)
`output()`	`this.picked.emit(2)`	`(picked)="handle($event)"`
`model()`	`this.value()`	`[(value)]="name"`
`output({alias})`	`this.picked.emit(2)`	`(choose)="handle($event)"`

Transforms and aliases

TypeScript

```ts
import { booleanAttribute, input, numberAttribute } from '@angular/core';

@Component({ selector: 'app-toggle', template: `` })
export class ToggleComponent {
  disabled = input(false, { transform: booleanAttribute });
  size = input(1, { transform: numberAttribute, alias: 'px-size' });
}
```

`booleanAttribute` turns the presence of the attribute into `true`, so `<app-toggle disabled>` works without `="true"`.

3.3 Examples

Example 1: child with one required input.

TypeScript

```ts
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-housing',
  template: `<p>{{ house().name }} - {{ house().city }}</p>`,
})
export class HousingComponent {
  house = input.required<{ name: string; city: string }>();
}
```

Example 2: parent template binds it.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<app-housing [house]="current" />
```

TypeScript

```ts
import { Component } from '@angular/core';

@Component({ selector: 'app-parent', templateUrl: './parent.html', imports: [HousingComponent] })
export class ParentComponent {
  current = { name: 'Acorn Cottage', city: 'Portland' };
}
```

Expected rendering: `Acorn Cottage - Portland`.

Example 3: child emits, parent listens.

TypeScript

```typescript
import { Component, output } from '@angular/core';

@Component({
  selector: 'app-rate',
  template: `<button type="button" (click)="rate(5)">Rate</button>`,
})
export class RateComponent {
  rated = output<number>();
  rate(value: number) {
    this.rated.emit(value);
  }
}
```

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<app-rate (rated)="lastRating = $event" />
<p>You rated: {{ lastRating }}</p>
```

Example 4: two-way binding with a model signal.

TypeScript

```typescript
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-slider',
  template: `<input type="range" [value]="level()" (input)="level.set($any($event.target).valueAsNumber
})
export class SliderComponent {
  level = model(50);
}
```

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<app-slider [(level)]="parentLevel" />
<p>Parent sees: {{ parentLevel }}</p>
```

`[(level)]="parentLevel"` is shorthand for `[level]="parentLevel"` plus `(levelChange)="parentLevel = $event"`, because a model signal exposes the output as the input name plus `Change`.

Example 5: a disabled button driven by a boolean input.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<button type="button" [disabled]="saving()">Save</button>
```

TypeScript

```typescript
import { Component, signal } from '@angular/core';

@Component({ selector: 'app-save', template: `<app-save-button [disabled]="saving()" />`, imports: [Sav
export class SaveComponent {
  saving = signal(false);
}
```

`[disabled]` sets the DOM property, so the value must be a boolean; the attribute form `disabled="false"` would leave the button disabled because the attribute is present.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Writing to an input from the child	breaks one-way data flow	emit an output or use `model()`
Reading `this.name` instead of `this.name()` in the class	you get the signal, not the value	call the input
Forgetting `[disabled]` brackets	binds a string, not the property	use property binding
Required inputs not bound	compile-time error, good	fix the binding, do not make it optional
Emitting inside the constructor	parent not ready yet	emit in response to user actions

3.5 Practice exercises

1. Declare an input named `label` of type `string` with no default.
2. Which decorator expression makes an input mandatory?
3. Rewrite `<app-x [level]="v" (levelChange)="v = $event">` with the short form.
4. How does a child tell the parent that an item was chosen?
5. What does `booleanAttribute` do in `input(false, { transform: booleanAttribute })`?

Solutions

1. `label = input<string>();` or `label = input('');`.
2. `input.required<string>()`.
3. `<app-x [(level)]="v" />`.
4. Declare `picked = output<Item>()` and call `this.picked.emit(item)`.
5. It converts the bound value so that the attribute's presence means `true` and its absence `false`, even when the value arrives as a string.

3.6 Mini test

1. Multiple choice: which call declares a required input? - a) `input.required()` - b) `requiredInput()` - c) `input({ required: true })` - d) `input.strict()`
2. Multiple choice: how does the template read a signal input? - a) `{{ label }}` - b) `{{ label() }}` - c) `{{ label.value }}` - d) `{{ label | async }}`
3. True or false: a child may assign to its own input.
4. Short answer: which method sends a value out of an output?
5. Multiple choice: what is `[(level)]="v"` short for? - a) `[level]="v"` - b) `(level)="v"` - c) `[level]="v" (levelChange)="v = $event"` - d) `[level]="v" (change)="v = $event"`

6. True or false: `[disabled]="true"` sets the DOM property.
7. Short code question: declare an output that emits a string.

Answer key and explanations

1. a - `input.required()`.
2. b - inputs are signals, so they are called.
3. False - inputs are read-only; use an output or a model.
4. `emit`, as in `this.picked.emit(value)`.
5. c - the two-way sugar expands to the input binding plus the `Change` event.
6. True - square brackets bind the property.
7. `nameChange = output<string>();`

Chapter 4 - Templates and control flow

4.1 Summary

- Templates render values from the class with interpolation `{{ }}`.
- Property binding writes to DOM properties; attribute, class and style bindings cover the rest of the DOM.
- `@if` and `@else` replace the old structural directive for conditionals.
- `@for` repeats a block and requires a `track` expression for stable identity.
- `@empty` renders a fallback when the list is empty.
- `@switch` and `@case` handle multi-branch selection.
- `@for` exposes `$index`, `$first`, `$last`, `$even` and `$odd`.
- Class and style bindings accept maps or single properties.

4.2 Concepts

Binding kinds

Syntax	Target	Example
`{{ expr }}`	text content	`{{ title }}`
`[prop]="expr"`	DOM property	`[src]="url"`
`[attr.name]="expr"`	DOM attribute	`[attr.aria-label]="hint"`
`[class.on]="expr"`	one class	`[class.active]="isActive"`
`[class]="map"`	many classes	`[class]="{ on: ok, off: !ok }"`
`[style.width.px]="expr"`	style with unit	`[style.width.px]="w"`
`(event)="stmt"`	event listener	`(click)="save()"`

Conditionals with `@if`

HTML

```
@if (items().length > 0) {
  <p>You have {{ items().length }} items.</p>
} @else if (loading()) {
  <p>Loading...</p>
} @else {
  <p>Nothing here yet.</p>
}
```

Loops with `@for`

HTML

```
@for (item of items(); track item.id) {
  <li [class.first]="$first">{{ $index }}. {{ item.name }}</li>
} @empty {
  <li>No results</li>
}
```

`track` is mandatory. Tracking by a stable id lets Angular reuse DOM nodes; tracking by `$index` is only safe for static lists.

Switch

HTML

```
@switch (status()) {
  @case ('loading') { <p>Loading...</p> }
  @case ('error') { <p>Something went wrong.</p> }
  @default { <p>Ready.</p> }
}
```

4.3 Examples

Example 1: render a list of locations.

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
<ul>
  @for (place of places(); track place.id) {
    <li>{{ place.city }} ({{ place.state }})</li>
  } @empty {
    <li>No places</li>
  }
</ul>
```

TypeScript

```ts
import { Component, signal } from '@angular/core';

@Component({ selector: 'app-places', templateUrl: './places.html' })
export class PlacesComponent {
  places = signal([
    { id: 1, city: 'Portland', state: 'OR' },
    { id: 2, city: 'Seattle', state: 'WA' },
  ]);
}
```

Expected rendering: two list items, `Portland (OR)` and `Seattle (WA)`.

Example 2: index and position flags.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
@for (name of names(); track name; let i = $index, first = $first) {
  <p>{{ i }} {{ name }} {{ first ? '(first)' : '' }}</p>
}
```

Example 3: a search box with a filtered count.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<input type="search" [value]="query()" (input)="query.set($any($event.target).value)" />
<p>{{ matches().length }} matches found</p>
```

Example 4: class and style bindings.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<button type="button" [class.active]="tab() === 'a'" [style.opacity]="tab() === 'a' ? 1 : 0.5">
  Tab A
</button>
```

Example 5: attribute binding for ARIA.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<button type="button" [attr.aria-expanded]="open()" (click)="open.set(!open())">
  Details
</button>
```

`[attr.aria-expanded]` writes the attribute as a string, so `open()` being `true` produces `aria-expanded="true"`.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`@for` without `track`	template compile error	always write `track`
`track $index` on editable lists	wrong DOM reuse, lost input state	track a stable id
Calling methods in bindings	runs every change detection	use `computed` or fields

Mistake	Why it hurts	Best practice
`{{ }}` with statements	compile error	use `@if` or a field
`[disabled]="'false'"`	truthy string disables the control	bind a real boolean
Deeply nested `@if` chains	unreadable templates	extract child components

4.5 Practice exercises

1. Write an `@for` over `tasks()` tracking `t.id` that shows `t.title`.
2. What does `@empty` need to follow?
3. Render `Loading` while `busy()` is true and `Done` otherwise.
4. Which loop variable gives the 0-based position?
5. Why does `[class.active]="isActive"` beat `class="{{ isActive ? 'active' : '' }}"`?

Solutions

1. `@for (t of tasks(); track t.id) { <p>{{ t.title }}</p> }`
2. An `@for` block in the same element.
3. `@if (busy()) { <p>Loading</p> } @else { <p>Done</p> }`
4. `$index`.
5. The property-style binding toggles one class without rebuilding the whole `class` attribute, and it cannot be fooled by a falsy-looking string.

4.6 Mini test

1. Multiple choice: which block is mandatory inside `@for`? - a) `@empty` - b) `track` - c) `let` - d) `@case`
2. Multiple choice: which variable is the item's position? - a) `$position` - b) `$index` - c) `$i` - d) `$order`
3. True or false: `@if` replaces `*ngIf` in new templates.
4. Short answer: which binding writes `aria-label` as an attribute?
5. Multiple choice: what does `[style.width.px]="w"` produce when `w` is 40? - a) `width="40"` - b) `style="width: 40px"` - c) `style="width: 40"` - d) nothing
6. True or false: interpolation accepts `if` statements.
7. Short code question: write `@switch` on `mode()` with a default branch.

Answer key and explanations

1. b - `track` is mandatory and states how items are identified.
2. b - `$index`.
3. True - built-in control flow is the current syntax.
4. `[attr.aria-label]="..."`.

5. b - the `.px` suffix adds the unit.
6. False - expressions only.
7. `html @switch (mode()) { @case ('edit') { <p>Edit</p> } @default { <p>View</p> } }`

Chapter 5 - Dependency injection and services

5.1 Summary

- Angular's injector creates services and hands them to whoever asks.
- `@Injectable({ providedIn: 'root' })` makes a service a singleton available everywhere.
- `inject()` obtains a dependency inside a class field initialiser or constructor.
- Providers can also be declared per component, which scopes the instance to that component subtree.
- An `InjectionToken` provides values that are not classes, such as config objects.
- The environment injector holds app-wide services; element injectors hold component-level providers.
- `providedIn: 'any'` and `providedIn: SomeModule` are deprecated; use `root`, `platform`, `null` or an explicit provider list.
- `injectAsync()` loads a service lazily and returns a function that resolves to it.

5.2 Concepts

A root service

TypeScript

```ts
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class HousingService {
  private locations = [
    { id: 0, name: 'Acorn Cottage', city: 'Portland' },
    { id: 1, name: 'Birch House', city: 'Seattle' },
  ];

  all() {
    return this.locations;
  }

  byId(id: number) {
    return this.locations.find((l) => l.id === id);
  }
}
```

Provider style	Scope	Lifetime
`providedIn: 'root'`	whole app	one instance

Provider style	Scope	Lifetime
`providedIn: 'platform'`	all apps on the page	one per platform
component `providers` array	that component's subtree	one per provider instance
route `providers` array	that route's components	one per route

Injecting

TypeScript

```ts
import { Component, inject } from '@angular/core';
import { HousingService } from './housing.service';

@Component({
  selector: 'app-list',
  template: `<p>{{ service.all().length }} places</p>`,
})
export class ListComponent {
  service = inject(HousingService);
}
```

A token for plain values

TypeScript

```ts
import { InjectionToken } from '@angular/core';

export interface ApiConfig {
  baseUrl: string;
  timeoutMs: number;
}

export const API_CONFIG = new InjectionToken<ApiConfig>('api.config');
```

TypeScript

```ts
import { ApplicationConfig } from '@angular/core';
import { API_CONFIG } from './api.config';

export const appConfig: ApplicationConfig = {
  providers: [{ provide: API_CONFIG, useValue: { baseUrl: '/api', timeoutMs: 5000 } }],
};
```

Scoping a service to a component

TypeScript

```
import { Component } from '@angular/core';
import { DraftService } from './draft.service';

@Component({
  selector: 'app-editor',
  providers: [DraftService],
  template: `<p>{{ draft.text }}</p>`,
})
export class EditorComponent {
  draft = inject(DraftService);
}
```

Every `EditorComponent` gets its own `DraftService`; removing the component destroys it.

5.3 Examples

Example 1: service plus component.

TypeScript

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class CounterStore {
  value = 0;
  add(n: number) {
    this.value += n;
  }
}
```

TypeScript

```
import { Component, inject } from '@angular/core';
import { CounterStore } from './counter-store.service';

@Component({
  selector: 'app-store-demo',
  template: `<button type="button" (click)="store.add(1)">{{ store.value }}</button>`,
})
export class StoreDemoComponent {
  store = inject(CounterStore);
}
```

Example 2: lazy service loading with `injectAsync`.

TypeScript

```
import { Component, injectAsync, onIdle } from '@angular/core';

@Component({
  selector: 'app-report',
  template: `<button type="button" (click)="load()">Load report</button>`,
})
export class ReportComponent {
  private reportService = injectAsync(() => import('./report.service'), { prefetch: onIdle });

  async load() {
    const service = await this.reportService();
    service.build();
  }
}
```

`onIdle` asks for the module while the browser is idle, so the click feels instant.

Example 3: injecting a token.

TypeScript

```ts
import { Component, Inject, inject } from '@angular/core';
import { API_CONFIG, ApiConfig } from './api.config';

@Component({ selector: 'app-api', template: `<p>{{ config.baseUrl }}</p>` })
export class ApiComponent {
  config: ApiConfig = inject(API_CONFIG);
}
```

Example 4: the same class injected twice at different scopes.

TypeScript

```ts
import { Component, inject } from '@angular/core';
import { DraftService } from './draft.service';

@Component({
  selector: 'app-page',
  providers: [DraftService],
  template: `
    <app-editor />
    <app-editor />
  `,
  imports: [EditorComponent],
})
export class PageComponent {
  pageDraft = inject(DraftService);
}
```

The page-level `DraftService` and the one each editor receives are three different instances, because the editors declared the provider themselves.

Example 5: no provider found.

TypeScript

```ts
import { Component, inject } from '@angular/core';
import { MissingService } from './missing.service';

@Component({ selector: 'app-broken', template: `` })
export class BrokenComponent {
  missing = inject(MissingService);
}
```

Expected behaviour: the application fails to start with error NG0201, which names the missing token. The fix is either `providedIn: 'root'` on the service or a provider entry.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`new HousingService()` in a component	breaks DI, loses the singleton	`inject(HousingService)`
Forgetting `@Injectable`	injector cannot build the class	add the decorator

Mistake	Why it hurts	Best practice
Root-provided state in many components	hidden shared state	keep state in one service
`providedIn: 'any'` for app singletons	new instance per lazy chunk	use `root`
Business logic in components	untestable components	logic in services

5.5 Practice exercises

1. Which decorator makes a class injectable at the root?
2. How do you obtain a service inside a component field?
3. Which provider style gives every component its own instance?
4. What is an `InjectionToken` for?
5. What does error NG0201 tell you?

Solutions

1. `@Injectable({ providedIn: 'root' })`.
2. `private svc = inject(MyService);`.
3. Listing the class in the component's `providers` array.
4. Providing values that are not classes, such as configuration objects.
5. That a requested dependency had no provider; the message names the token.

5.6 Mini test

1. Multiple choice: which function asks the injector for a dependency? - a) `resolve` - b) `inject` - c) `provide` - d) `instantiate`
2. Multiple choice: which value of `providedIn` creates one app-wide instance? - a) `'any'` - b) `'root'` - c) `'platform'` - d) `null`
3. True or false: a component can provide its own instance of a root service.
4. Short answer: which class do you use to inject a plain configuration object?
5. Multiple choice: what happens when a dependency has no provider? - a) it is `undefined` - b) error NG0201 is thrown - c) the component is skipped - d) nothing
6. True or false: `injectAsync` returns the service itself.
7. Short code question: inject a service named `AuthService` into a field.

Answer key and explanations

1. b - `inject(Token)`.
2. b - `'root'`.
3. True - the component `providers` array shadows the root provider for that subtree.
4. `InjectionToken<T>`.

5. b - NG0201 names the missing token.
6. False - it returns a function that resolves to the service.
7. `private auth = inject(AuthService);`

Chapter 6 - Routing and navigation

6.1 Summary

- The router maps URL paths to components and keeps the URL in sync.
- `provideRouter(routes)` registers the route table in the application config.
- `<router-outlet />` marks where the matched component is rendered.
- `routerLink` on an anchor performs navigation without a page reload.
- Route parameters such as `details/:id` come from `ActivatedRoute.paramMap`.
- `Router.navigate` navigates from code, which is useful after an action.
- A wildcard route with `**` and `redirectTo` handles unknown URLs.
- `loadComponent` loads a route's code only when the route is first visited.

6.2 Concepts

The route table

TypeScript

```ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home';
import { DetailsComponent } from './details/details';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'details/:id', component: DetailsComponent },
  { path: 'search', loadComponent: () => import('./search/search').then((m) => m.SearchComponent) },
  { path: '**', redirectTo: '' },
];
```

The shell

TypeScript

```ts
import { Component } from '@angular/core';
import { RouterLink, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  template: `
    <nav>
      <a routerLink="/">Home</a>
      <a routerLink="/details/0">First listing</a>
    </nav>
    <router-outlet />
  `,
  imports: [RouterLink, RouterOutlet],
})
export class AppComponent {}
```

Reading parameters

TypeScript

```ts
import { Component, inject, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-details',
  template: `<p>Listing {{ id }}</p>`,
})
export class DetailsComponent implements OnInit {
  private route = inject(ActivatedRoute);
  id = 0;

  ngOnInit() {
    this.route.paramMap.subscribe((params) => {
      this.id = Number(params.get('id'));
    });
  }
}
```

`paramMap` is an observable that emits on every parameter change, so navigating from `details/0` to `details/1` reuses the same component instance.

Navigating from code

TypeScript

```ts
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({ selector: 'app-save-done', template: `` })
export class SaveDoneComponent {
  private router = inject(Router);

  goHome() {
    this.router.navigate(['/']);
  }
}
```

6.3 Examples

Example 1: two static routes and a fallback.

TypeScript

```ts
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '**', redirectTo: '' },
];
```

Example 2: links in the template.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<nav>
  <a routerLink="/">Home</a>
  <a routerLink="/about">About</a>
  <a [routerLink]="['/details', item.id]">Details of {{ item.name }}</a>
</nav>
```

The third form binds an array, so segments can contain expressions.

Example 3: query parameters.

TypeScript

```ts
this.router.navigate(['/search'], { queryParams: { city: 'Portland' } });
```

Example 4: reading the query string.

TypeScript

```ts
import { Component, inject, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({ selector: 'app-search', template: `<p>City: {{ city }}</p>` })
export class SearchComponent implements OnInit {
  private route = inject(ActivatedRoute);
  city = '';

  ngOnInit() {
    this.route.queryParamMap.subscribe((q) => (this.city = q.get('city') ?? ''));
  }
}
```

Example 5: a lazy route.

TypeScript

```ts
{
  path: 'details/:id',
  loadComponent: () => import('./details/details').then((m) => m.DetailsComponent),
}
```

Expected behaviour: the browser downloads this component's code the first time the URL matches, not at startup.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`<a href="/details/0">`	full page reload	`routerLink`
Missing `router-outlet`	matched route never shows	add it to the shell
No wildcard route	blank page on a bad URL	`{ path: '**', redirectTo: '' }`
Subscribing without unsubscribing in long-lived code	leaks	use the `takeUntilDestroyed` pattern or async pipe
Navigating before checking validity	user lands on an error page	guard the route or check first
Duplicate path prefixes	the first match wins, later routes never run	order static paths before dynamic ones

6.5 Practice exercises

1. Which provider registers the route table?
2. Write the route for `details/:id` that lazy-loads its component.
3. Which directive renders the matched component?
4. How does the template navigate to `/about` without a reload?
5. Which observable gives the current value of `:id`?

Solutions

1. `provideRouter(routes)`.
2. `{ path: 'details/:id', loadComponent: () => import('./details/details').then((m) => m.DetailsComponent) }`.
3. `<router-outlet />` (the `RouterOutlet` directive).
4. `<a routerLink="/about">About</a>`.
5. `ActivatedRoute.paramMap`.

6.6 Mini test

1. Multiple choice: which call registers routes? - a) `provideRoutes` - b) `provideRouter` - c) `useRoutes` - d) `RouterModule.forRoot`
2. Multiple choice: which directive marks the outlet? - a) `router-view` - b) `ng-outlet` - c) `router-outlet` - d) `router-host`
3. True or false: `paramMap` emits again when only the parameter changes.
4. Short answer: which property of a route loads code on demand?
5. Multiple choice: what does `{ path: '**', redirectTo: '' }` do? - a) deletes the URL - b) sends unknown URLs to the home route - c) blocks navigation - d) reloads the page
6. True or false: `<a href>` and `routerLink` behave the same.

7. Short code question: navigate to `['/details', 7]` from a method.

Answer key and explanations

1. b - `provideRouter(routes)`.
2. c - `router-outlet`.
3. True - the component is reused and the observable re-emits.
4. `loadComponent`.
5. b - it is the wildcard fallback.
6. False - `href` reloads the page, `routerLink` does not.
7. `this.router.navigate(['/details', 7]);`

Chapter 7 - Forms: template-driven and reactive

7.1 Summary

- Angular has two form models: template-driven, built on `ngModel` in the template, and reactive, built on `FormControl` and `FormGroup` in the class.
- `FormsModule` gives you template-driven forms; `ReactiveFormsModule` gives you reactive forms.
- A `FormControl` tracks its value, validity, touched and dirty state.
- Validators such as `Validators.required` and `Validators.email` live in the class, where they are easy to test.
- A `FormGroup` combines controls and can carry a group-level validator.
- `valueChanges` and `statusChanges` emit on every edit and status change.
- `reset()` restores the initial state; `disable()` takes a control out of validation and the submitted value.
- Reactive forms are the default choice for anything beyond a single field.

7.2 Concepts

Reactive building blocks

```
FormGroup { email: FormControl, password: FormControl }
   |
   +-- .value          { email: 'a@b.c', password: 'secret' }
   +-- .valid / .status
   +-- .errors         null when valid
   +-- .touched /.dirty
```

A group with validators

TypeScript

```ts
import { FormControl, FormGroup, Validators } from '@angular/forms';

export class LoginForm {
  form = new FormGroup({
    email: new FormControl('', [Validators.required, Validators.email]),
    password: new FormControl('', [Validators.required, Validators.minLength(8)]),
  });

  submit() {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
      return;
    }
    console.log(this.form.value);
  }
}
```

The template

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<form [formGroup]="form" (ngSubmit)="submit()">
  <label>
    Email
    <input type="email" formControlName="email" />
  </label>
  @if (form.controls.email.touched && form.controls.email.invalid) {
    <p class="error">Enter a valid email address.</p>
  }
  <label>
    Password
    <input type="password" formControlName="password" />
  </label>
  <button type="submit" [disabled]="form.invalid">Sign in</button>
</form>
```

Template-driven alternative

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<input [(ngModel)]="name" name="name" required minlength="2" />
<p>{{ name }}</p>
```

`name` is mandatory so Angular can register the control with the form.

7.3 Examples

Example 1: a single control's state.

TypeScript

```ts
import { FormControl, Validators } from '@angular/forms';

const name = new FormControl('', Validators.required);

console.log('status       :', name.status);
console.log('hasError     :', name.hasError('required'), name.errors?.required);
console.log('touched/dirty:', name.touched, name.dirty);

name.setValue('Ada');
console.log('after set    :', name.status, name.hasError('required'), name.errors);
```

output

```
status       : INVALID
hasError     : true true
touched/dirty: false false
after set    : VALID false null
```

`errors` is `null` while the control is valid; an empty string fails `required`, so the control starts `INVALID`.

Example 2: the email validator.

TypeScript

```ts
import { FormControl, Validators } from '@angular/forms';

const email = new FormControl('ada@example.com', [Validators.required, Validators.email]);
console.log('email ok     :', email.status, email.errors);

email.setValue('not-an-email');
console.log('email bad    :', email.status, JSON.stringify(email.errors));
```

output

```
email ok     : VALID null
email bad    : INVALID {"email":true}
```

Example 3: a group with a cross-field validator.

TypeScript

```ts
import { FormControl, FormGroup, Validators } from '@angular/forms';

const passwords = new FormGroup(
  {
    password: new FormControl('', Validators.minLength(8)),
    confirm: new FormControl(''),
  },
  (control) =>
    control.get('password')!.value === control.get('confirm')!.value ? null : { mismatch: true },
);

console.log('group empty  :', passwords.status, JSON.stringify(passwords.errors));
passwords.setValue({ password: 'longenough', confirm: 'longenough' });
console.log('group match  :', passwords.status, JSON.stringify(passwords.errors));
passwords.patchValue({ confirm: 'different' });
console.log('group differ :', passwords.status, JSON.stringify(passwords.errors));
console.log('group value  :', JSON.stringify(passwords.value));
```

output

```
group empty  : VALID null
group match  : VALID null
group differ : INVALID {"mismatch":true}
group value  : {"password":"longenough","confirm":"different"}
```

An empty group is `VALID` because `minLength` skips empty values; combine it with `required` when a value is mandatory.

Example 4: reacting to edits.

TypeScript

```ts
import { FormControl } from '@angular/forms';

const name = new FormControl('');
name.valueChanges.subscribe((v) => console.log('valueChanges :', JSON.stringify(v)));
name.statusChanges.subscribe((s) => console.log('statusChanges:', s));

name.setValue('Grace');
name.reset();
console.log('after reset  :', name.status, JSON.stringify(name.value), name.touched, name.dirty);
```

output

```
valueChanges : "Grace"
statusChanges: VALID
valueChanges : null
statusChanges: INVALID
after reset  : INVALID null false false
```

`reset()` with no argument clears the value and returns to pristine.

Example 5: typed group values in TypeScript.

TypeScript

```ts
import { FormControl, FormGroup, Validators } from '@angular/forms';

interface Login {
  email: string;
  password: string;
}

const form = new FormGroup<Login>({
  email: new FormControl('', { nonNullable: true, validators: [Validators.required] }),
  password: new FormControl('', { nonNullable: true, validators: [Validators.required] }),
});
```

The generic keeps `form.value` typed as `Login` instead of a partial object.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`ngModel` without `name`	control is not registered	always add `name`
Disabling the submit button only	no feedback on what is wrong	show field errors too

Mistake	Why it hurts	Best practice
Validators in the template for complex rules	untestable and verbose	reactive validators in the class
`minLength` alone for a required field	empty value passes	add `required`
Forgetting to unsubscribe from `valueChanges`	leaks in long-lived subjects	use `takeUntilDestroyed` or destroy them with the owner
Reading `form.value` while invalid	sends broken data	check `form.invalid` first

7.5 Practice exercises

1. Which module do you import for template-driven forms?
2. Create a `FormControl` for a required age with value `0`.
3. Why does an empty group with only `minLength(8)` stay valid?
4. Which method marks every control as touched?
5. What do `valueChanges` and `statusChanges` emit?

Solutions

1. `FormsModule`.
2. `new FormControl(0, Validators.required)`.
3. `minLength` ignores empty values, so validation must include `required` to demand input.
4. `form.markAllAsTouched()`.
5. `valueChanges` emits the new value on each edit; `statusChanges` emits `VALID`, `INVALID` or `PENDING`.

7.6 Mini test

1. Multiple choice: which class holds one field's value and state? - a) `FormArray` - b) `FormControl` - c) `FormGroup` - d) `FormModel`
2. Multiple choice: which module provides `formControlName`? - a) `FormsModule` - b) `ReactiveFormsModule` - c) `CommonModule` - d) `RouterModule`
3. True or false: `form.errors` is an empty object when the group is valid.
4. Short answer: which validator demands a non-empty value?
5. Multiple choice: what does `reset()` do? - a) destroys the group - b) clears values and resets touched and dirty - c) re-runs validators only - d) submits the form
6. True or false: a group-level validator receives the whole group.
7. Short code question: build a group with `email` and `password`, both required.

Answer key and explanations

1. b - `FormControl`.

2. b - `ReactiveFormsModule`.
3. False - it is `null` when there are no errors.
4. `Validators.required`.
5. b - it restores the initial state.
6. True - the factory receives the group and returns errors or `null`.
7. `ts const form = new FormGroup({ email: new FormControl('', Validators.required), password: new FormControl('', Validators.required), });`

Chapter 8 - HTTP client and async data access

8.1 Summary

- `HttpClient` sends requests and returns observables of parsed responses.
- `provideHttpClient(withFetch())` registers the client in the application config.
- `get<T>()`, `post<T>()`, `put<T>()` and `delete<T>()` set the response type so the compiler checks it.
- An observable only runs when something subscribes to it.
- RxJS operators such as `map`, `catchError` and `switchMap` shape the stream.
- `firstValueFrom` turns a one-shot request into a promise.
- HTTP failures arrive as `HttpErrorResponse` with `status` and `statusText`.
- Requests live in services, not in components.

8.2 Concepts

Registering the client

TypeScript

```ts
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withFetch } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient(withFetch())],
};
```

A typed data service

TypeScript

```typescript
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface HousingLocation {
  id: number;
  name: string;
  city: string;
}

@Injectable({ providedIn: 'root' })
export class HousingService {
  private http = inject(HttpClient);
  private baseUrl = '/api/locations';

  all(): Observable<HousingLocation[]> {
    return this.http.get<HousingLocation[]>(this.baseUrl);
  }

  byId(id: number): Observable<HousingLocation> {
    return this.http.get<HousingLocation>(`${this.baseUrl}/${id}`);
  }

  create(location: Omit<HousingLocation, 'id'>): Observable<HousingLocation> {
    return this.http.post<HousingLocation>(this.baseUrl, location);
  }
}
```

Consuming it in a component

TypeScript

```typescript
import { Component, inject, OnInit } from '@angular/core';
import { HousingService, HousingLocation } from './housing.service';

@Component({
  selector: 'app-list',
  template: `
    @if (error(); as message) {
      <p class="error">{{ message }}</p>
    }
    @for (place of places(); track place.id) {
      <p>{{ place.city }}</p>
    }
  `,
})
export class ListComponent implements OnInit {
  private service = inject(HousingService);
  places = signal<HousingLocation[]>([]);
  error = signal('');

  ngOnInit() {
    this.service.all().subscribe({
      next: (data) => this.places.set(data),
      error: (err: HttpErrorResponse) => this.error.set(err.message),
    });
  }
}
```

Error handling

TypeScript

```ts
import { catchError, map, throwError } from 'rxjs';
import { HttpErrorResponse } from '@angular/common/http';

all() {
  return this.http.get<HousingLocation[]>(this.baseUrl).pipe(
    map((rows) => rows.filter((r) => r.city.length > 0)),
    catchError((error: HttpErrorResponse) => {
      console.error(error.status, error.statusText);
      return throwError(() => new Error('Could not load the listings'));
    }),
  );
}
```

`map` transforms the payload; `catchError` replaces the failing stream; the subscriber's `error` callback receives the new error.

8.3 Examples

Example 1: the request/response cycle.

```
component               service                 backend
   |  all()                |                       |
   | --------------------> |  GET /api/locations    |
   |                       | ---------------------> |
   |                       | <--------------------- |  200 + JSON
   | <-------------------- |  Observable<Location[]>|
   | subscribe: places.set |                       |
```

Example 2: a search that cancels the previous request.

TypeScript

```ts
import { Component, inject } from '@angular/core';
import { switchMap } from 'rxjs';
import { HousingService } from './housing.service';

@Component({ selector: 'app-search', template: `` })
export class SearchComponent {
  private service = inject(HousingService);
  private terms = new Subject<string>();

  results$ = this.terms.pipe(switchMap((term) => this.service.search(term)));

  search(term: string) {
    this.terms.next(term);
  }
}
```

`switchMap` drops the previous request's result when a new term arrives, so slow responses cannot overwrite fresh ones.

Example 3: posting with `firstValueFrom`.

TypeScript

```
import { firstValueFrom } from 'rxjs';

async add() {
  const created = await firstValueFrom(
    this.service.create({ name: 'New listing', city: 'Portland' }),
  );
  console.log(created.id);
}
```

Example 4: a component that shows loading state.

TypeScript

```
import { Component, inject, signal } from '@angular/core';
import { HousingService } from './housing.service';

@Component({ selector: 'app-one', template: `<p>{{ place()?.city }}</p>` })
export class OneComponent {
  private service = inject(HousingService);
  place = signal<{ city: string } | null>(null);

  load() {
    this.service.byId(0).subscribe((data) => this.place.set(data));
  }
}
```

Example 5: what a failure looks like.

TypeScript

```
import { HttpErrorResponse } from '@angular/common/http';

this.service.byId(999).subscribe({
  error: (error: HttpErrorResponse) => {
    console.log(error.status, error.statusText, error.message);
  },
});
```

Expected behaviour for a missing resource: the error callback runs, `status` is the server's code (for example `404`) and `statusText` describes it. The exact values come from the server, so they are not reproduced here.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting to subscribe	the request never leaves the browser	subscribe or use `firstValueFrom`
HTTP calls inside components	duplicated logic in templates	keep them in services
Untyped `get()`	`any` spreads through the app	`get<T>()` with an interface
Ignoring the error channel	silent failures	always provide `error`
Subscribing in a loop	stacked requests	`switchMap` or a single subscription

Mistake	Why it hurts	Best practice
Raw URLs scattered in components	hard to change	one `baseUrl` in the service

8.5 Practice exercises

1. Which function registers the HTTP client?
2. Why does `this.http.get('/api')` do nothing on its own?
3. Which operator keeps only the latest search request?
4. What does `catchError` return in the example above?
5. Which type describes an HTTP failure?

Solutions

1. `provideHttpClient(withFetch())`, listed in the application providers.
2. Observables are lazy: without a subscriber no request is sent.
3. `switchMap`.
4. A new observable created by `throwError(...)`, so the subscriber's error callback receives the wrapped error.
5. `HttpErrorResponse`.

8.6 Mini test

1. Multiple choice: which service sends HTTP requests? - a) `FetchService` - b) `HttpClient` - c) `HttpRouter` - d) `RequestService`
2. Multiple choice: which method sets the response type? - a) `get<Location[]>()` - b) `get(Location[])` - c) `get().of(Location)` - d) `getAs(Location)`
3. True or false: every observable runs as soon as it is created.
4. Short answer: which RxJS function converts a request observable into a promise?
5. Multiple choice: where should the `baseUrl` live? - a) in each component - b) in the service - c) in the template - d) in `index.html`
6. True or false: `catchError` can replace the whole failing stream.
7. Short code question: subscribe with separate `next` and `error` callbacks.

Answer key and explanations

1. b - `HttpClient`, injected into a service.
2. a - the generic sets the response type.
3. False - observables are lazy and need a subscriber.
4. `firstValueFrom`.
5. b - one service owns the URL.
6. True - it returns a replacement observable.

7. `this.service.all().subscribe({ next: (d) => ..., error: (e) => ... });`

Chapter 9 - Signals: state, computed, linked and effects

9.1 Summary

- A signal is a function that returns a value and records who read it.
- `signal()` creates writable state; `computed()` derives a value; both are read by calling them.
- Writing the same value to a signal notifies nobody, because equality is `Object.is` by default.
- `computed()` memoizes: the computation runs again only when a dependency changes.
- `linkedSignal()` is a writable signal whose value follows a source signal until you set it.
- `effect()` runs a side effect when the signals it reads change; it must be created in an injection context.
- `model()` declares a two-way signal for component communication.
- `viewChild()` and `contentChild()` are signal queries for elements and child components.
- `untracked()` reads a signal without recording a dependency.

9.2 Concepts

Creating and reading

TypeScript

```ts
import { signal, computed } from '@angular/core';

const price = signal(10);
const qty = signal(3);
const total = computed(() => price() * qty());

console.log('total      :', total());
price.set(20);
console.log('total after:', total());
```

output

```
total      : 30
total after: 60
```

Writable signal API

TypeScript

```
const count = signal(0);

count.set(5);                 // replace
count.update((c) => c + 1);   // derive from the previous value
const frozen = count.asReadonly();
```

Equality and options

TypeScript

```
const point = signal(
  { x: 1, y: 2 },
  { equal: (p, q) => p.x === q.x && p.y === q.y, debugName: 'point' },
);
```

A custom `equal` prevents downstream work when a new object is equivalent.

linkedSignal

TypeScript

```
import { linkedSignal, signal } from '@angular/core';

const options = signal(['a', 'b', 'c']);
const selectedId = signal('a');
const selected = linkedSignal(() => options().find((o) => o === selectedId()) ?? null);
```

The value recomputes whenever `options` or `selectedId` changes; after a manual `set`, the next source change overwrites it again.

effect

TypeScript

```
import { Component, effect, signal } from '@angular/core';

@Component({ selector: 'app-logger', template: `` })
export class LoggerComponent {
  count = signal(0);

  constructor() {
    effect(() => console.log('count is', this.count()));
  }
}
```

Effects run during change detection, after the DOM has been updated. Signal writes are allowed inside effects in v22, but writing a signal that the same effect reads makes it loop, and Angular stops the application with an error.

Signal queries

TypeScript

```
import { Component, viewChild, contentChild, ElementRef } from '@angular/core';

@Component({ selector: 'app-box', template: `<input #field />` })
export class BoxComponent {
  field = viewChild.required<ElementRef>('field');
}
```

`viewChild.required` throws when the element is missing; the plain `viewChild` returns `undefined`. `contentChild` looks at projected content instead of the component's own template.

9.3 Examples

Example 1: writing to a signal.

TypeScript

```ts
import { signal } from '@angular/core';

const count = signal(0);
console.log('initial    :', count());
count.set(5);
count.update((c) => c + 1);
console.log('after set/update:', count());
```

output

```
initial    : 0
after set/update: 6
```

Example 2: memoization.

TypeScript

```ts
import { computed, signal } from '@angular/core';

let calls = 0;
const a = signal(1);
const b = signal(10);
const sum = computed(() => {
  calls++;
  return a() + b();
});

console.log('sum       :', sum(), 'computations:', calls);
console.log('sum again :', sum(), 'computations:', calls);
b.set(20);
console.log('sum new   :', sum(), 'computations:', calls);
b.set(20);
console.log('sum same  :', sum(), 'computations:', calls);
```

output

```
sum       : 11 computations: 1
sum again : 11 computations: 1
sum new   : 21 computations: 2
sum same  : 21 computations: 2
```

Reading twice costs nothing, and writing an equal value changes nothing.

Example 3: `untracked`.

TypeScript

```ts
import { computed, signal, untracked } from '@angular/core';

let calls = 0;
const tracked = signal(1);
const notTracked = signal(100);
const mixed = computed(() => {
  calls++;
  return tracked() + untracked(() => notTracked());
});

console.log('mixed      :', mixed(), 'calls:', calls);
notTracked.set(200);
console.log('mixed again:', mixed(), 'calls:', calls);
```

output

```
mixed      : 101 calls: 1
mixed again: 101 calls: 1
```

The untracked read still sees the current value, but it does not make the computed stale.

Example 4: linked signal behaviour.

TypeScript

```ts
import { linkedSignal, signal } from '@angular/core';

const items = signal(['apple', 'banana']);
const selectedIndex = signal(0);
const selected = linkedSignal(() => items()[selectedIndex()] ?? null);

console.log('selected   :', selected());
selectedIndex.set(1);
console.log('selected   :', selected());
selected.set('cherry');
console.log('after set  :', selected());
items.set(['kiwi']);
console.log('after reset:', selected());
```

output

```
selected   : apple
selected   : banana
after set  : cherry
after reset: null
```

The last line is `null` because the index is still `1` while the new list has one item, so the lookup fails and the fallback applies.

Example 5: where `model()` and `effect()` may be created.

TypeScript

```ts
import { model, signal } from '@angular/core';

const outside = model('');
```

Expected behaviour: this throws runtime error NG0203, because `model()` needs an injection context. The same is true for `input()`, `output()` and `effect()` outside a component, directive or service field initialiser. `signal()` and `computed()` work anywhere.

Example 6: a model inside a component.

TypeScript

```typescript
import { Component, model } from '@angular/core';

@Component({ selector: 'app-toggle', template: `<button type="button">{{ label() }}</button>` })
export class ToggleComponent {
  label = model('off');
}
```

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`this.count` instead of `this.count()` in the class	you read the signal object	call it
Mutating a signal's object in place	no change notification	set a new object
Writing a signal an effect reads	infinite loop, Angular stops the app	derive state with `computed` instead
Creating `effect()` in a plain function	NG0203	create it in a class field
Heavy work inside `computed`	runs again whenever a dependency changes	keep computations cheap
Using signals for one-off events	state, not events	use `output()` or a service

9.5 Practice exercises

1. Which function creates writable signal state?
2. How many times does the computation of Example 2 run after the four logs?
3. What happens when you write the same value twice in a row?
4. Which function reads a signal without tracking it?
5. Why does `linkedSignal` reset?

Solutions

1. `signal(initialValue)`.
2. Twice: once for the first read, once after `b.set(20)`.
3. Nothing: the default equality check sees no change and notifies nobody.
4. `untracked(() => ...)`.
5. Because its value is derived from a source: any source change recomputes it, which overwrites manual writes.

9.6 Mini test

1. Multiple choice: which call creates derived state? - a) `signal` - b) `computed` - c) `effect` - d) `model`
2. Multiple choice: how do you read a signal in a class? - a) `this.count` - b) `this.count()` - c) `this.count.value` - d) `this.count.get()`
3. True or false: a `computed` recomputes on every read.
4. Short answer: which error appears when `effect()` is created outside an injection context?
5. Multiple choice: which signal is writable and follows a source? - a) `computed` - b) `linkedSignal` - c) `model.required` - d) `viewChild`
6. True or false: `model()` works as both an input and an output.
7. Short code question: derive a `double` signal from `count`.

Answer key and explanations

1. b - `computed`.
2. b - signals are functions.
3. False - it memoizes until a dependency changes.
4. NG0203.
5. b - `linkedSignal`.
6. True - it exposes the value plus a `Change` output.
7. `const double = computed(() => count() * 2);`

Chapter 10 - Deferrable views and image optimization

10.1 Summary

- `@defer` keeps a block of a template out of the initial render until a trigger fires.
- `@placeholder` shows static content before the block starts loading.
- `@loading` shows content while the block's code loads, with `minimum` and `after` timing options.
- `@error` shows content when loading fails.
- Triggers are `on viewport`, `on idle`, `on interaction`, `on hover`, `on timer`, `on immediate` and `when`.
- `prefetch` loads the code before the trigger fires, without rendering.
- `NgOptimizedImage` replaces `src` with `ngSrc` and requires `width` and `height` so the browser can reserve space.
- `priority` marks above-the-fold images, and a `placeholder` URL gives a low-quality preview.

10.2 Concepts

The four blocks

HTML

```
@defer (on viewport) {
  <app-reviews [listingId]="id()" />
} @placeholder {
  <p>Reviews appear here.</p>
} @loading (after 200ms; minimum 400ms) {
  <p>Loading reviews...</p>
} @error {
  <p>Reviews could not be loaded.</p>
}
```

`@placeholder` content must not depend on the deferred block. `after 200ms` delays the loading message so it does not flash; `minimum 400ms` keeps it visible long enough to read.

Triggers

Trigger	Fires when
`on viewport`	the placeholder enters the visible area
`on idle`	the browser is idle
`on interaction`	the user clicks or taps the placeholder
`on hover`	the pointer enters the placeholder
`on timer(500ms)`	500 milliseconds pass
`on immediate`	after the initial render finishes
`when condition`	the condition becomes true
`prefetch on idle`	prefetches the code; rendering waits for the main trigger

Several triggers can be combined; the first one wins.

Images

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
<img [ngSrc]="photo()" width="800" height="600" alt="Front of the house" priority />
```

`ngSrc` is a property, so the value must be bound with brackets. The `width` and `height` attributes let the browser compute the aspect ratio before the image arrives, which avoids layout shift. `priority` raises the fetch priority and disables lazy loading for that image, which suits the first image on a page.

10.3 Examples

Example 1: defer until the user scrolls.

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
<section>
  <h2>Listing</h2>
  @defer (on viewport; prefetch on idle) {
    <app-map [location]="place()" />
  } @placeholder {
    <div class="map-skeleton"></div>
  }
</section>
```

Expected behaviour: the map's code is fetched while the browser is idle, and the block renders once the placeholder scrolls into view.

Example 2: defer behind a click.

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
@defer (on interaction) {
  <app-comments />
} @placeholder {
  <button type="button">Show comments</button>
}
```

Example 3: defer with a condition.

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
<button type="button" (click)="showDetails.set(true)">Details</button>
@defer (when showDetails()) {
  <app-details />
}
```

Example 4: an optimized image list.

HTML

```
<!-- NOTE: browser-only rendering; not executed here. -->
@for (photo of photos(); track photo.id) {
  <img [ngSrc]="photo.url" width="400" height="300" alt="{{ photo.alt }}" />
}
```

Example 5: importing the directive.

TypeScript

```
import { Component } from '@angular/core';
import { NgOptimizedImage } from '@angular/common';

@Component({
  selector: 'app-gallery',
  templateUrl: './gallery.html',
  imports: [NgOptimizedImage],
})
export class GalleryComponent {}
```

`NgOptimizedImage` is a directive, so a standalone component must import it before using `ngSrc`.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`@placeholder` that calls deferred code	defeats the purpose	keep it static and cheap
`@for` inside `@defer` without `track`	same compile error as elsewhere	keep `track` on every loop
`src` instead of `ngSrc`	no optimization, layout shift	`ngSrc` with `width` and `height`
Missing `width` and `height`	layout shift when the image loads	declare both
`priority` on every image	competes with critical resources	only the first visible image
Deferring everything	many small requests	defer heavy, below-the-fold blocks

10.5 Practice exercises

1. Which block shows content before the deferred code loads?
2. Write a `@defer` that waits for a click on its placeholder.
3. Which attribute replaces `src` with `NgOptimizedImage`?
4. Why do images need `width` and `height`?
5. What does `prefetch` do?

Solutions

1. `@placeholder`.
2. `html @defer (on interaction) { <app-comments /> } @placeholder { <button type="button">Show comments</button> }`
3. `ngSrc`, bound as `[ngSrc]`.
4. So the browser can reserve the right space before the bytes arrive and avoid layout shift.
5. It downloads the deferred block's code early without rendering it, so the actual trigger feels instant.

10.6 Mini test

1. Multiple choice: which block renders while the deferred code loads? - a) `@placeholder` - b) `@loading` - c) `@error` - d) `@empty`
2. Multiple choice: which trigger waits for the browser to be idle? - a) `on timer` - b) `on idle` - c) `on immediate` - d) `on hover`
3. True or false: `@defer` keeps the code out of the initial bundle download.
4. Short answer: which timing option on `@loading` delays the message?
5. Multiple choice: what does `priority` do on an image? - a) sorts the list - b) raises fetch priority and skips lazy loading - c) defers the image - d) compresses the image

6. True or false: `ngSrc` can be written as a plain attribute `ngSrc="photo.jpg"`.
7. Short code question: write a `@defer` on viewport with a placeholder paragraph.

Answer key and explanations

1. b - `@loading`.
2. b - `on idle`.
3. True - the block is loaded on demand.
4. `after`, as in `@loading (after 200ms)`.
5. b - it prioritises the fetch for above-the-fold images.
6. True - static values work as an attribute; dynamic values need `[ngSrc]="expr"`.
7. `html @defer (on viewport) { <app-heavy /> } @placeholder { <p>Coming soon</p> }`

Chapter 11 - Pipes and data formatting

11.1 Summary

- A pipe transforms a value inside an interpolation or binding.
- Built-in pipes cover dates, numbers, currency, percentages, case changes, slicing and JSON.
- Pipes take parameters after a colon and can be chained.
- `@Pipe({ name })` declares a custom pipe whose class has a `transform` method.
- Pipes are pure by default: they re-run only when the input reference changes.
- `AsyncPipe` subscribes to an observable or promise and unwraps the value.
- `KeyValuePipe` turns an object into key/value pairs, sorted by key by default.
- Formatting belongs in the template, not in the data class.

11.2 Concepts

Syntax

HTML

```
1  {{ value | pipeName:arg1:arg2 }}
```

Pipe	Example	Purpose	
`date`	`{{ today	date:'fullDate' }}`	format dates
`currency`	`{{ 1234.5	currency:'USD' }}`	money
`number`	`{{ 3.14159	number:'1.1-2' }}`	digits
`percent`	`{{ 0.256	percent:'1.1-1' }}`	percentages

Pipe	Example	Purpose	
`uppercase`, `lowercase`, `titlecase`	`{{ 'ada'	titlecase }}`	case
`slice`	`{{ list	slice:0:3 }}`	arrays and strings
`json`	`{{ obj	json }}`	debugging output
`keyvalue`	`{{ map	keyvalue }}`	object to pairs
`async`	`{{ data$	async }}`	unwrap streams

A custom pipe

TypeScript

```ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'fileSize' })
export class FileSizePipe implements PipeTransform {
  transform(bytes: number, digits = 1): string {
    if (bytes === 0) {
      return '0 B';
    }
    const units = ['B', 'KB', 'MB', 'GB', 'TB'];
    const exponent = Math.min(Math.floor(Math.log(bytes) / Math.log(1024)), units.length - 1);
    return `${(bytes / 1024 ** exponent).toFixed(digits)} ${units[exponent]}`;
  }
}
```

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<p>{{ 5242880 | fileSize }}</p>
<p>{{ 5242880 | fileSize:3 }}</p>
```

Expected rendering: `5.0 MB` and `5.000 MB`.

Purity

A pure pipe runs only when its input changes by reference, which keeps templates fast. Marking a pipe `pure: false` makes it run on every change detection cycle and is only right for values that mutate without changing reference.

11.3 Examples

Example 1: the built-in formatting pipes, executed with `@angular/common` 22.2.2.

TypeScript

```
import { CurrencyPipe, DatePipe, DecimalPipe, PercentPipe, SlicePipe, TitleCasePipe, UpperCasePipe } fr

const datePipe = new DatePipe('en-US');
console.log('DatePipe short    :', datePipe.transform(new Date(2026, 9, 10)));
console.log('DatePipe fullDate :', datePipe.transform(new Date(2026, 9, 10), 'fullDate'));
console.log('DatePipe medium   :', datePipe.transform(new Date(2026, 9, 10), 'medium'));

const decimal = new DecimalPipe('en-US');
console.log('DecimalPipe 3.1-4 :', decimal.transform(3.14159, '3.1-4'));
console.log('DecimalPipe 1.0-2 :', decimal.transform(2.5, '1.0-2'));

const currency = new CurrencyPipe('en-US');
console.log('CurrencyPipe USD  :', currency.transform(1234.5, 'USD', 'symbol'));
console.log('CurrencyPipe code :', currency.transform(1234.5, 'USD', 'code'));

const percent = new PercentPipe('en-US');
console.log('PercentPipe 1.2-2 :', percent.transform(0.256, '1.2-2'));

const upper = new UpperCasePipe();
console.log('UpperCasePipe     :', upper.transform('ada lovelace'));

const title = new TitleCasePipe();
console.log('TitleCasePipe     :', title.transform('ADA LOVELACE'));

const slice = new SlicePipe();
console.log('SlicePipe 0:3     :', slice.transform([1, 2, 3, 4, 5], 0, 3));
console.log('SlicePipe string  :', slice.transform('Angular', 0, 6));
```

output

```
DatePipe short    : Oct 10, 2026
DatePipe fullDate : Saturday, October 10, 2026
DatePipe medium   : Oct 10, 2026, 12:00:00 AM
DecimalPipe 3.1-4 : 003.1416
DecimalPipe 1.0-2 : 2.5
CurrencyPipe USD  : $1,234.50
CurrencyPipe code : USD1,234.50
PercentPipe 1.2-2 : 25.60%
UpperCasePipe     : ADA LOVELACE
TitleCasePipe     : Ada Lovelace
SlicePipe 0:3     : [ 1, 2, 3 ]
SlicePipe string  : Angula
```

The date output depends on the locale passed to the pipe class; the template form `{{ value | date }}` uses the application's locale, which defaults to `en-US`.

Example 2: the custom pipe's logic, executed as a plain function.

TypeScript

```ts
function fileSize(bytes: number, digits = 1): string {
  if (bytes === 0) {
    return '0 B';
  }
  const units = ['B', 'KB', 'MB', 'GB', 'TB'];
  const exponent = Math.min(Math.floor(Math.log(bytes) / Math.log(1024)), units.length - 1);
  return `${(bytes / 1024 ** exponent).toFixed(digits)} ${units[exponent]}`;
}

console.log(fileSize(0));
console.log(fileSize(512));
console.log(fileSize(2048));
console.log(fileSize(5_242_880));
console.log(fileSize(5_242_880, 3));
```

output

```
0 B
512.0 B
2.0 KB
5.0 MB
5.000 MB
```

Example 3: chaining and parameters.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<p>{{ 'ada lovelace' | titlecase | uppercase }}</p>
<p>{{ 1234.5 | currency:'USD':'symbol':'1.0-0' }}</p>
```

Example 4: `AsyncPipe`.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
@if (listing$ | async; as listing) {
  <h2>{{ listing.name }}</h2>
}
```

The pipe subscribes when the view appears and unsubscribes when it is destroyed, so the component does not manage the subscription.

Example 5: `keyvalue`.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
@for (row of { b: 2, a: 1 } | keyvalue; track row.key) {
  <p>{{ row.key }}: {{ row.value }}</p>
}
```

`keyvalue` needs Angular's change detection to read object changes, so unlike the pure formatting pipes above it cannot be exercised outside a running application.

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Formatting in the class	formatting logic spreads everywhere	use pipes in the template
Forgetting the locale-sensitive default	surprising output for other regions	pass format strings explicitly
Mutating an array in place	pure pipe does not re-run	replace the array
`pure: false` for convenience	runs on every cycle	fix the reference instead
Pipes for filtering large lists in `@for`	recomputation cost	filter in the class or a `computed`
Unsubscribed observables in components	leaks	`AsyncPipe`

11.5 Practice exercises

1. Which pipe formats `0.256` as a percentage?
2. Write a custom pipe named `excerpt` that cuts text at `limit` characters and adds an ellipsis.
3. Why does a pure pipe not re-run after an in-place array sort?
4. What does `AsyncPipe` do with a subscription?
5. Which pipe prints an object as JSON for debugging?

Solutions

1. `percent`, as in `{{ 0.256 | percent }}`.
2. `ts @Pipe({ name: 'excerpt' }) export class ExcerptPipe implements PipeTransform { transform(text: string, limit = 40): string { return text.length <= limit ? text : ${text.slice(0, limit).trimEnd()}...; } }` With `limit = 20`, the input `A comfortable room in the middle of the city` produces `"A comfortable room i..."`.
3. Because the array reference did not change, so the pipe sees the same input.
4. It subscribes for the lifetime of the view and unsubscribes on destroy.
5. `json`.

11.6 Mini test

1. Multiple choice: which pipe formats money? - a) `number` - b) `currency` - c) `money` - d) `price`
2. Multiple choice: which separator introduces pipe parameters? - a) `,` - b) `:` - c) `=` - d) `-`
3. True or false: pipes may be chained.
4. Short answer: which decorator declares a custom pipe?
5. Multiple choice: what is the default purity of a pipe? - a) pure - b) impure - c) chosen by the CLI - d) none
6. True or false: `AsyncPipe` manages subscriptions for you.
7. Short code question: display `price` with two decimals.

Answer key and explanations

1. b - `currency`.
2. b - a colon starts the parameter list.
3. True - the left pipe's output feeds the right pipe.
4. `@Pipe({ name: '...' })`.
5. a - pure by default.
6. True - it subscribes and unsubscribes with the view.
7. `{{ price | number:'1.2-2' }}`.

Chapter 12 - Signal-based forms

12.1 Summary

- Signal forms wrap a signal model: the model is the single source of truth.
- `form(model)` returns a field tree that mirrors the shape of the model.
- Field state is read by calling the field, then the state signal: `state.valid()`, `state.touched()`, `state.errors()`.
- Rules are attached with functions such as `required`, `email`, `minLength`, `max`, `pattern`, `validate` and `validateHttp`.
- The `FormField` directive with the `[formField]` attribute connects a control to a field.
- `submit(form, options)` validates, runs the action and reports success.
- Errors are objects with a `kind` and an optional `message`.
- `schema()` builds reusable rule sets, and `applyEach` applies them to array items.

12.2 Concepts

The model and the form

TypeScript

```ts
import { Component, signal } from '@angular/core';
import { form, required, email, minLength } from '@angular/forms/signals';

@Component({
  selector: 'app-login',
  imports: [FormField],
  template: `
    <form (submit)="send($event)">
      <label>
        Email
        <input type="email" [formField]="loginModel.email" />
      </label>
      <button type="submit">Sign in</button>
    </form>
  `,
})
export class LoginComponent {
  private model = signal({ email: '', password: '' });

  loginModel = form(this.model, (f) => {
    required(f.email);
    email(f.email);
    minLength(f.password, 8);
  });

  send(event: Event) {
    event.preventDefault();
  }
}
```

The signal forms live in their own entry point, `@angular/forms/signals`, kept separate from the classic reactive API.

Reading field state

TypeScript

```ts
const emailState = this.loginModel.email();

emailState.value();      // the current value
emailState.valid();      // no errors and nothing pending
emailState.invalid();    // at least one error
emailState.touched();    // the user has focused and blurred the control
emailState.dirty();      // the value changed
emailState.errors();     // the error objects
```

`valid()` and `invalid()` are not opposites: while an asynchronous validator is still pending, `valid()` is `false` but `invalid()` is also `false`.

Standard rules

Rule	Applies to
`required(path)`	any field
`email(path)`	string fields
`minLength(path, n)`, `maxLength(path, n)`	strings and arrays
`min(path, n)`, `max(path, n)`	numbers

Rule	Applies to
`minDate`, `maxDate`	dates
`pattern(path, re)`	strings
`validate(path, fn)`	custom sync logic
`validateHttp(path, opts)`	server-side checks
`disabled(path)`, `hidden(path)`, `readonly(path)`	state rules

Submitting

TypeScript

```ts
import { submit } from '@angular/forms/signals';

async send() {
  const ok = await submit(this.loginModel, {
    action: async (field) => {
      console.log(field().value());
    },
    onInvalid: () => console.log('fix the errors first'),
  });
  console.log('submitted:', ok);
}
```

`action` runs only when the form is valid and must return a promise; `submit` resolves to `true` on success.

12.3 Examples

Example 1: error display in the template.

HTML

```html
<!-- NOTE: browser-only rendering; not executed here. -->
<label>
  Email
  <input type="email" [formField]="loginModel.email" />
</label>
@if (loginModel.email().touched() && loginModel.email().invalid()) {
  <ul class="errors">
    @for (error of loginModel.email().errors(); track error.kind) {
      <li>{{ error.message ?? error.kind }}</li>
    }
  </ul>
}
```

Example 2: a custom message.

TypeScript

```ts
required(f.email, { message: 'We need an email address.' });
minLength(f.password, 8, { message: 'Use at least 8 characters.' });
```

Example 3: conditional logic.

TypeScript

```
form(this.model, (f) => {
  required(f.name);
  required(f.email);
  applyWhenValue(f.contact, (kind) => kind === 'email', (f2) => email(f2));
});
```

Example 4: an array of items.

TypeScript

```
const guestSchema = schema<{ name: string }>((guest) => {
  required(guest.name);
});

form(this.guests, (guests) => {
  applyEach(guests, guestSchema);
});
```

Example 5: what the API looks like from the type side.

TypeScript

```
import { form, required } from '@angular/forms/signals';
import { signal } from '@angular/core';

// NOTE: this needs an injection context, so it is shown as it appears in a
// class field; it was not executed in this book's Node sandbox.
const loginForm = form(signal({ first: '', last: '' }), (name) => {
  required(name.first);
});
console.log(loginForm.first().value());
```

The field tree is callable: calling the tree gives the field state, and calling a state member such as `value()` gives the current value.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Importing `form` from `@angular/forms`	symbol not found; it lives in `@angular/forms/signals`	import from `@angular/forms/signals`
Keeping a second copy of the data	two sources of truth drift	the model signal is the data
Forgetting `[formField]`	the control is not connected	bind the field
Showing errors before the user types	hostile interface	check `touched()` first
Reading `state.errors` without calling it	you get a signal object	call the signal
Long rule lists inline	unreadable	extract a `schema()`

12.5 Practice exercises

1. Which entry point exports `form()`?
2. How do you read a field's validity?
3. Which function attaches a validator that runs on the server?
4. What does `submit()` return?
5. How do you reuse one rule set for every item of an array field?

Solutions

1. `@angular/forms/signals`.
2. Call the field, then the state: `form.email().valid()`.
3. `validateHttp(path, options)`.
4. A promise of a boolean: `true` when the action ran.
5. Build it with `schema<Shape>((item) => { ... })` and apply it with `applyEach(arrayField, itemSchema)`.

12.6 Mini test

1. Multiple choice: which function creates the field tree? - a) `new FormGroup` - b) `form(model)` - c) `createForm` - d) `formGroup(model)`
2. Multiple choice: which attribute binds a control to a field? - a) `formControlName` - b) `formField` - c) `formModel` - d) `control`
3. True or false: the model signal stays the source of truth after the form is created.
4. Short answer: which state signal tells you the user has left the control?
5. Multiple choice: what happens when `submit()` runs on an invalid form? - a) it throws - b) `onInvalid` runs and no action runs - c) the action runs anyway - d) the form resets
6. True or false: `valid()` and `invalid()` are exact opposites.
7. Short code question: make the `email` field of `f` required with a custom message.

Answer key and explanations

1. b - `form(model, schema?)`.
2. b - the `FormField` directive selector `[formField]`.
3. True - updates to the field write back to the model.
4. `touched`.
5. b - the action is skipped and `onInvalid` reports it.
6. False - with a pending async validator both are `false`.
7. `required(f.email, { message: 'Email is required.' });`

Cheat sheet

13.1 CLI

shell

```shell
npm install -g @angular/cli     # install once
ng new study-app --routing      # scaffold
ng generate component name      # add a component
ng generate service name        # add a service
ng serve --open                 # dev server, port 4200
ng build                         # production bundle
```

13.2 Component

TypeScript

```typescript
@Component({
  selector: 'app-x',
  template: `<p>{{ value }}</p>`,
  styleUrl: './x.css',
  imports: [ChildComponent],
})
export class XComponent {
  value = 'text';               // state
  save() { ... }                // behaviour
}
```

13.3 Inputs, outputs, models

TypeScript

```typescript
name = input('');                       // optional input, signal
must = input.required<string>();        // required input
flag = input(false, { transform: booleanAttribute });
size = input(1, { alias: 'px-size' });  // renamed public API
picked = output<number>();              // emit with picked.emit(2)
level = model(0);                       // two-way with [(level)]
```

HTML

```html
<app-child [name]="name" (picked)="handle($event)" [(level)]="v" />
```

13.4 Control flow

HTML

```
@if (a()) { ... } @else if (b()) { ... } @else { ... }
@for (item of items(); track item.id) { {{ item.name }} } @empty { none }
@switch (mode()) { @case ('a') { ... } @default { ... } }
```

13.5 Bindings

HTML

```
{{ expr }}                      [prop]="expr"          [attr.aria-x]="expr"
(click)="do()"                  [class.on]="ok"         [style.width.px]="w"
```

13.6 Dependency injection

TypeScript

```ts
@Injectable({ providedIn: 'root' })
export class S { ... }

@Injectable()
export class Scoped { ... }      // list in component providers

class C { s = inject(S); }       // token works too: inject(API_CONFIG)
```

13.7 Router

TypeScript

```ts
providers: [provideRouter([
  { path: '', component: Home },
  { path: 'details/:id', loadComponent: () => import('./d').then(m => m.D) },
  { path: '**', redirectTo: '' },
])]
```

HTML

```html
<a routerLink="/about">About</a>
<router-outlet />
```

TypeScript

```ts
route = inject(ActivatedRoute);
this.route.paramMap.subscribe((p) => p.get('id'));
inject(Router).navigate(['/details', 7]);
```

13.8 Reactive forms

TypeScript

```ts
form = new FormGroup({
  email: new FormControl('', [Validators.required, Validators.email]),
  password: new FormControl('', Validators.minLength(8)),
}, (g) => g.get('password')!.value === g.get('confirm')!.value ? null : { mismatch: true });
```

HTML

```html
<form [formGroup]="form" (ngSubmit)="save()">
  <input formControlName="email" />
  <button type="submit" [disabled]="form.invalid">Go</button>
</form>
```

13.9 Signals

TypeScript

```
count = signal(0);                    // set/update/asReadonly
double = computed(() => count() * 2);
picker = linkedSignal(() => items()[selected()] ?? null);
name = model('');                     // two-way
field = viewChild.required<ElementRef>('f');

constructor() { effect(() => console.log(this.count())); }
untracked(() => other());             // read without tracking
```

13.10 Defer and images

HTML

```
@defer (on viewport; prefetch on idle) { <app-heavy /> }
@placeholder { <p>placeholder</p> } @loading (after 200ms) { <p>loading</p> } @error { <p>error</p> }

<img [ngSrc]="url()" width="800" height="600" alt="Front" priority />
```

13.11 Pipes

HTML

```
{{ today | date:'fullDate' }}
{{ 1234.5 | currency:'USD' }}
{{ 0.256 | percent:'1.1-1' }}
{{ 'ada' | titlecase }}
{{ list | slice:0:3 }}
{{ data$ | async }}
{{ { a: 1 } | keyvalue }}
```

TypeScript

```
@Pipe({ name: 'fileSize' })
export class FileSizePipe implements PipeTransform {
  transform(bytes: number, digits = 1) { ... }
}
```

13.12 Signal forms

TypeScript

```
model = signal({ email: '', password: '' });
loginForm = form(this.model, (f) => {
  required(f.email, { message: 'Required' });
  email(f.email);
  minLength(f.password, 8);
});

state = this.loginForm.email();       // state.valid(), .touched(), .errors()
await submit(this.loginForm, { action: async (field) => call(field().value()) });
```

HTML

```
<input type="email" [formField]="loginForm.email" />
```

13.13 HTTP

TypeScript

```ts
providers: [provideHttpClient(withFetch())]

@Injectable({ providedIn: 'root' })
export class Api {
  http = inject(HttpClient);
  all(): Observable<Item[]> { return this.http.get<Item[]>('/api/items'); }
}
```

TypeScript

```ts
this.api.all().subscribe({ next: (d) => ..., error: (e: HttpErrorResponse) => ... });
await firstValueFrom(this.api.all());
```

Final test

1. Multiple choice: which call starts a standalone application? - a) `bootstrapModule` - b) `bootstrapApplication` - c) `ngServe` - d) `platformBrowser`
2. Multiple choice: which `@Component` field names the element? - a) `template` - b) `selector` - c) `styleUrl` - d) `imports`
3. True or false: component templates may contain `if` statements.
4. Multiple choice: how is a signal input read inside a component class? - a) `this.name` - b) `this.name()` - c) `this.name.value` - d) `this.name.get()`
5. Short answer: which function sends a value out of an `output()`?
6. Multiple choice: what expands `[(level)]="v"` into? - a) `[level]="v"` - b) `[level]="v" (levelChange)="v = $event"` - c) `(level)="v"` - d) `[level]="v" (change)="v"`
7. Multiple choice: which block is mandatory in an `@for`? - a) `@empty` - b) `track` - c) `let` - d) `@case`
8. Short code question: loop over `tasks()` tracking `t.id` and print `t.title`.
9. Multiple choice: which provider creates one app-wide service instance? - a) `providedIn: 'any'` - b) `providedIn: 'root'` - c) `providedIn: 'platform'` - d) no decorator
10. True or false: a component can give its subtree its own instance of a root-provided service.
11. Multiple choice: which directive renders the matched route? - a) `router-host` - b) `router-outlet` - c) `ng-view` - d) `router-view`
12. Multiple choice: which provider locks routes into the app? - a) `provideRoutes` - b) `provideRouter` - c) `RouterModule` - d) `useRoutes`
13. Short answer: which observable of `ActivatedRoute` reports `:id` changes?
14. Multiple choice: which module supplies `formControlName`? - a) `FormsModule` - b) `ReactiveFormsModule` - c) `CommonModule` - d) `RouterModule`
15. True or false: `Validators.minLength(8)` alone rejects an empty input.
16. Multiple choice: what does a group-level validator return when the group is valid? - a) `{}` - b) `null` - c) `undefined` - d) `true`

17. Multiple choice: which operator keeps only the newest HTTP result in a search? - a) `map` - b) `switchMap` - c) `catchError` - d) `tap`
18. True or false: an `HttpClient` observable sends its request without a subscriber.
19. Short answer: which type describes a failed HTTP request?
20. Multiple choice: how many times does a `computed` re-run when a dependency is set to the same value? - a) once - b) zero - c) twice - d) on every read
21. Multiple choice: which signal is writable and follows a source? - a) `computed` - b) `linkedSignal` - c) `input` - d) `output`
22. Short answer: which error code appears when `effect()` is created outside an injection context?
23. Multiple choice: which `@defer` block loads before the trigger fires? - a) `@placeholder` - b) `prefetch` - c) `@loading` - d) `@error`
24. Multiple choice: which attribute replaces `src` for optimized images? - a) `imgSrc` - b) `ngSrc` - c) `optimizedSrc` - d) `lazySrc`
25. Short code question: pipe `0.256` into a percentage with one decimal.

14.1 Final test answers

1. b - `bootstrapApplication(AppComponent, config)` is the standalone entry point.
2. b - `selector` names the custom element.
3. False - only expressions; conditionals use `@if`.
4. b - signal inputs are read by calling them.
5. `emit`, for example `this.picked.emit(2)`.
6. b - two-way binding adds the `Change` event.
7. b - `track` states how items are identified.
8. `html @for (t of tasks(); track t.id) { <p>{{ t.title }}</p> }`
9. b - `providedIn: 'root'`.
10. True - a component `providers` array shadows the root provider for its subtree.
11. b - `router-outlet`.
12. b - `provideRouter(routes)`.
13. `ActivatedRoute.paramMap`.
14. b - `ReactiveFormsModule`.
15. False - empty values skip `minLength`, so combine it with `required`.
16. b - `null` means no errors.
17. b - `switchMap` cancels the previous inner observable.
18. False - observables are lazy; a subscriber is required.
19. `HttpErrorResponse`.
20. b - zero, because the default equality check sees no change.
21. b - `linkedSignal`.
22. NG0203.

23. b - `prefetch` downloads the code without rendering.
24. b - `ngSrc`.
25. `{{ 0.256 | percent:'1.1-1' }}`, which renders `25.6%`.

Sources

Every URL below was consulted while planning and writing this volume. No text, images or code blocks were copied from these pages; all explanations, examples, exercises, tests and answers were written from scratch, and only short standard API names are shared with the source material.

Tutorial pages that define the syllabus for this volume:

- https://angular.dev/tutorials
- https://angular.dev/tutorials/learn-angular
- https://angular.dev/tutorials/learn-angular/1-components-in-angular
- https://angular.dev/tutorials/first-app
- https://angular.dev/tutorials/first-app/01-hello-world
- https://angular.dev/tutorials/signals
- https://angular.dev/tutorials/signals/1-creating-your-first-signal
- https://angular.dev/tutorials/deferrable-views
- https://angular.dev/tutorials/deferrable-views/1-what-are-deferrable-views
- https://angular.dev/tutorials/signal-forms
- https://angular.dev/tutorials/signal-forms/1-set-up-form-model
- https://angular.dev/ai/ai-tutor - the optional AI-tutor companion mentioned in the tutorial hub, not used for this volume

Reference pages whose API names were cross-checked against the shipped type definitions of the installed packages:

- https://angular.dev/guide/signals
- https://angular.dev/guide/signals/effect
- https://angular.dev/guide/signals/linked-signal
- https://angular.dev/guide/components/inputs
- https://angular.dev/guide/components/outputs
- https://angular.dev/guide/templates/two-way-binding
- https://angular.dev/guide/di/dependency-injection-context
- https://angular.dev/guide/di/lazy-loading-services
- https://angular.dev/guide/forms/signals/overview
- https://angular.dev/guide/forms/signals/models
- https://angular.dev/guide/forms/signals/validation
- https://angular.dev/guide/forms/signals/schemas

- https://angular.dev/guide/forms/signals/form-submission
- https://v22.angular.dev/errors/NG0201 and NG0203 - the runtime error pages quoted in the chapters on dependency injection and signals

npm packages installed on the authoring machine to execute the examples and read the type definitions, all at version 22.2.2:

- https://www.npmjs.com/package/@angular/core
- https://www.npmjs.com/package/@angular/common
- https://www.npmjs.com/package/@angular/forms
- https://www.npmjs.com/package/@angular/compiler